

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt

df = pd.read_csv('resources/survey_results.csv')
df.head()
```

	respondent_id	age	gender	zone	occupation	income_levels
0	R00001	30	M	Urban	Working Professional	<10L
1	R00002	46	F	Metro	Working Professional	> 35L
2	R00003	41	F	Rural	Working Professional	> 35L
3	R00004	33	F	Urban	Working Professional	16L - 25L
4	R00005	23	M	Metro	Student	NaN

	consume_frequency(weekly)	current_brand	preferable_consumption_size
0	3-4 times	Newcomer	Medium (500 ml)
1	5-7 times	Established	Medium (500 ml)
2	3-4 times	Newcomer	Medium (500 ml)
3	5-7 times	Newcomer	Medium (500 ml)
4	3-4 times	Established	Medium (500 ml)

	awareness_of_other_brands	reasons_for_choosing_brands
0	0 to 1	Price
1	2 to 4	Quality
2	2 to 4	Availability
3	0 to 1	Brand Reputation
4	0 to 1	Availability

	purchase_channel	packaging_preference	health_concerns
0	Online	Simple	Medium (Moderately health-conscious)
1	Retail Store	Premium	Medium (Moderately health-conscious)

```
conscious)
2      Retail Store          Premium  Medium (Moderately health-
conscious)
3          Online          Eco-Friendly          Low (Not very
concerned)
4          Online          Premium  Medium (Moderately health-
conscious)

typical_consumption_situations price_range
0      Active (eg. Sports, gym)    100-150
1          Social (eg. Parties)    200-250
2      Active (eg. Sports, gym)    200-250
3      Active (eg. Sports, gym)    150-200
4      Active (eg. Sports, gym)     50-100

df.shape
(30010, 17)
```

Data Cleaning

Remove Duplicates

```
df[df.duplicated()]

respondent_id age gender zone occupation \
2309      R02309  27      M   Urban  Working Professional
2666      R02665  61      M   Metro      Entrepreneur
5151      R05149  21      M Semi-Urban      Student
7794      R07791  25      M   Metro  Working Professional
8516      R08512  20      F   Metro      Student
11317     R11312  18      F   Urban      Student
19050     R19044  57      M Semi-Urban      Retired
22412     R22405  26      M   Metro  Working Professional
23406     R23398  27      M   Urban  Working Professional
25068     R25059  34      F   Urban  Working Professional

income_levels consume_frequency(weekly) current_brand \
2309      16L - 25L      3-4 times      Newcomer
2666      16L - 25L      3-4 times      Established
5151      NaN      3-4 times      Established
7794      <10L      3-4 times      Newcomer
8516      NaN      5-7 times      Established
11317     NaN      0-2 times      Newcomer
19050     <10L      3-4 times      Newcomer
22412     16L - 25L      0-2 times      Newcomer
```

23406	10L - 15L	5-7 times	Newcomer
25068	16L - 25L	0-2 times	Newcomer
preferable_consumption_size awareness_of_other_brands \			
2309	Small (250 ml)	2 to 4	
2666	Medium (500 ml)	above 4	
5151	Small (250 ml)	2 to 4	
7794	Medium (500 ml)	2 to 4	
8516	Large (1 L)	0 to 1	
11317	Medium (500 ml)	2 to 4	
19050	Large (1 L)	2 to 4	
22412	Small (250 ml)	0 to 1	
23406	Large (1 L)	above 4	
25068	Small (250 ml)	2 to 4	
reasons_for_choosing_brands flavor_preference			
purchase_channel \			
2309	Brand Reputation	Traditional	Online
2666	Brand Reputation	Exotic	Retail Store
5151	Availability	Traditional	Retail Store
7794	Availability	Exotic	Online
8516	Price	Exotic	Online
11317	Availability	Exotic	Online
19050	Price	Traditional	Retail Store
22412	Price	Exotic	Online
23406	Availability	Traditional	Online
25068	Brand Reputation	Exotic	Online
packaging_preference health_concerns \			
2309	Simple	Medium (Moderately health-conscious)	
2666	Simple	High (Very health-conscious)	
5151	Simple	Low (Not very concerned)	
7794	Simple	Medium (Moderately health-conscious)	
8516	Simple	Medium (Moderately health-conscious)	
11317	Premium	Medium (Moderately health-conscious)	
19050	Premium	High (Very health-conscious)	
22412	Premium	Low (Not very concerned)	
23406	Premium	High (Very health-conscious)	
25068	Eco-Friendly	High (Very health-conscious)	
typical_consumption_situations price_range			

2309	Social (eg. Parties)	150-200
2666	Active (eg. Sports, gym)	200-250
5151	Active (eg. Sports, gym)	50-100
7794	Casual (eg. At home)	100-150
8516	Casual (eg. At home)	100-150
11317	Social (eg. Parties)	100-150
19050	Social (eg. Parties)	150-200
22412	Active (eg. Sports, gym)	150-200
23406	Casual (eg. At home)	200-250
25068	Active (eg. Sports, gym)	150-200

```
print("No. of duplicate rows: ", len(df[df.duplicated()]))
```

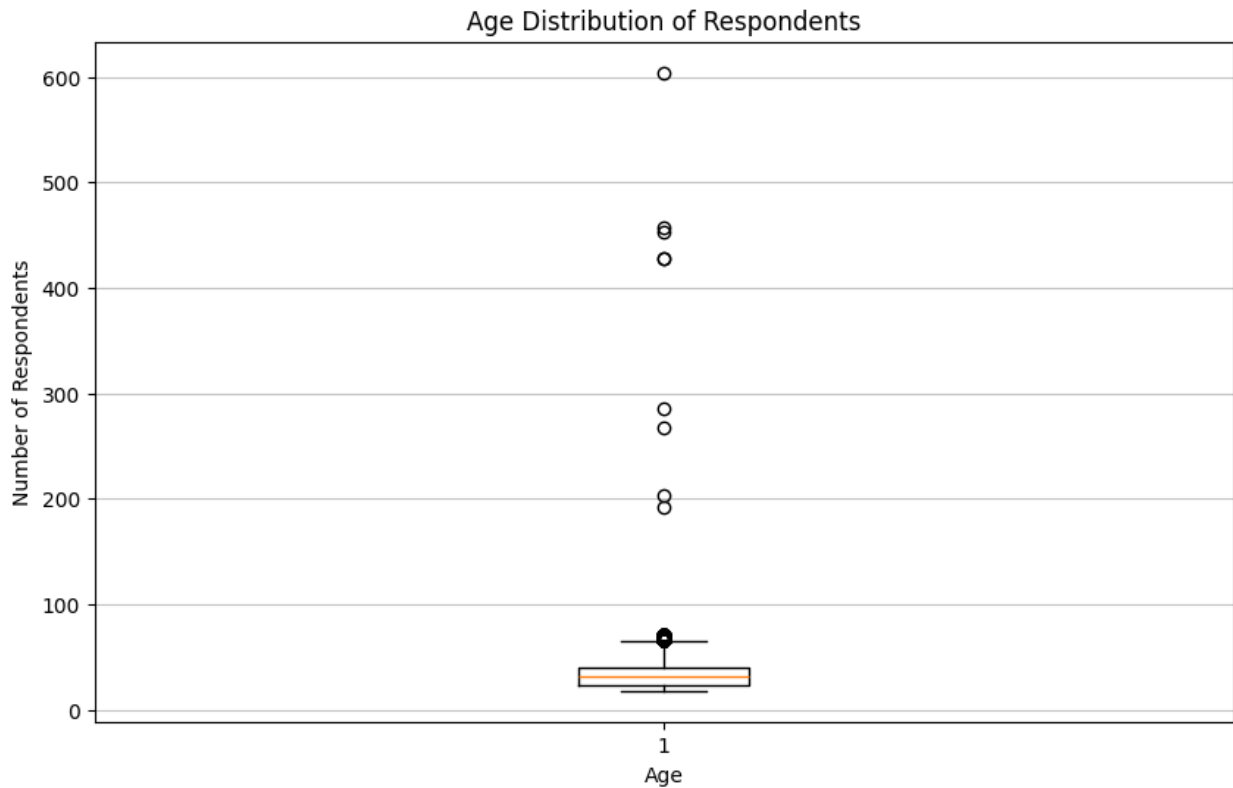
```
No. of duplicate rows: 10
```

```
df = df.drop_duplicates()
df.shape
```

```
(30000, 17)
```

Outliers in Age column

```
plt.figure(figsize=(10,6))
plt.boxplot(df['age'].dropna())
plt.title('Age Distribution of Respondents')
plt.xlabel('Age')
plt.ylabel('Number of Respondents')
plt.grid(axis='y', alpha=0.75)
plt.show()
```



```
df['age'].max()
```

```
np.int64(604)
```

```
df[df['age'] > 70]
```

	respondent_id	age	gender	zone	occupation	\
3099	R03098	453	M	Metro	Working Professional	
6262	R06260	428	M	Urban	Entrepreneur	
12403	R12398	604	M	Metro	Retired	
22549	R22542	457	M	Metro	Working Professional	
22918	R22911	267	F	Metro	Working Professional	
24958	R24950	285	M	Semi-Urban	Working Professional	
24960	R24952	192	F	Urban	Student	
25105	R25096	203	M	Metro	Working Professional	
28770	R28761	428	F	Rural	Working Professional	

	income_levels	consume_frequency(weekly)	current_brand	\
3099	26L - 35L	3-4 times	Established	
6262	16L - 25L	5-7 times	Established	
12403	<10L	0-2 times	Newcomer	
22549	16L - 25L	3-4 times	Newcomer	
22918	16L - 25L	3-4 times	Established	
24958	> 35L	0-2 times	Newcomer	
24960	NaN	3-4 times	Newcomer	
25105	16L - 25L	0-2 times	Established	

28770	26L - 35L	0-2 times	Established
	preferable_consumption_size	awareness_of_other_brands	\
3099	Medium (500 ml)	2 to 4	
6262	Large (1 L)	above 4	
12403	Small (250 ml)	2 to 4	
22549	Small (250 ml)	2 to 4	
22918	Medium (500 ml)	above 4	
24958	Small (250 ml)	2 to 4	
24960	Medium (500 ml)	0 to 1	
25105	Small (250 ml)	2 to 4	
28770	Small (250 ml)	2 to 4	
	reasons_for_choosing_brands	flavor_preference	
	purchase_channel	\	
3099	Brand Reputation	Traditional	Retail Store
6262	Quality	Exotic	Online
12403	Availability	Traditional	Online
22549	Price	Traditional	Online
22918	Brand Reputation	Exotic	Online
24958	Price	Traditional	Online
24960	Price	Traditional	Online
25105	Brand Reputation	Exotic	Retail Store
28770	Brand Reputation	Exotic	Online
	packaging_preference	health_concerns	\
3099	Premium	Medium (Moderately health-conscious)	
6262	Simple	High (Very health-conscious)	
12403	Simple	Medium (Moderately health-conscious)	
22549	Premium	High (Very health-conscious)	
22918	Eco-Friendly	Low (Not very concerned)	
24958	Simple	Medium (Moderately health-conscious)	
24960	Simple	Low (Not very concerned)	
25105	Premium	High (Very health-conscious)	
28770	Simple	Medium (Moderately health-conscious)	
	typical_consumption_situations	price_range	
3099	Social (eg. Parties)	200-250	
6262	Social (eg. Parties)	200-250	
12403	Casual (eg. At home)	100-150	
22549	Casual (eg. At home)	200-250	

22918	Active (eg. Sports, gym)	150-200
24958	Active (eg. Sports, gym)	150-200
24960	Active (eg. Sports, gym)	50-100
25105	Active (eg. Sports, gym)	150-200
28770	Casual (eg. At home)	150-200

```
print(len(df[df['age'] > 100]))
print(len(df[df['age'] > 70]))
```

```
9
9
```

```
q1 = df['age'].quantile(0.25)
q3 = df['age'].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
print("Lower Bound: ", lower_bound)
print("Upper Bound: ", upper_bound)
```

```
Lower Bound: -2.5
Upper Bound: 65.5
```

```
df = df[df['age'] <= 70]
df.shape
```

```
(29991, 17)
```

Correct spelling

```
columns_list = ['gender', 'zone', 'occupation', 'income_levels',
                'consume_frequency(weekly)', 'current_brand',
                'preferable_consumption_size', 'awareness_of_other_brands',
                'reasons_for_choosing_brands', 'flavor_preference',
                'purchase_channel',
                'packaging_preference', 'health_concerns',
                'typical_consumption_situations', 'price_range']
```

```
unique_values = {col: df[col].unique().tolist() for col in
columns_list}
unique_values
```

```
{'gender': ['M', 'F'],
 'zone': ['Urban', 'Metro', 'Rural', 'Semi-Urban', 'Metor', 'urbna'],
 'occupation': ['Working Professional', 'Student', 'Entrepreneur',
'Retired'],
 'income_levels': ['<10L',
 '> 35L',
 '16L - 25L',
 nan,
```

```

    '10L - 15L',
    '26L - 35L'],
    'consume_frequency(weekly)': ['3-4 times', '5-7 times', '0-2 times',
nan],
    'current_brand': ['Newcomer', 'Established', 'newcomer',
'Establishd'],
    'preferable_consumption_size': ['Medium (500 ml)',
    'Large (1 L)',
    'Small (250 ml)'],
    'awareness_of_other_brands': ['0 to 1', '2 to 4', 'above 4'],
    'reasons_for_choosing_brands': ['Price',
    'Quality',
    'Availability',
    'Brand Reputation'],
    'flavor_preference': ['Traditional', 'Exotic'],
    'purchase_channel': ['Online', 'Retail Store', nan],
    'packaging_preference': ['Simple', 'Premium', 'Eco-Friendly'],
    'health_concerns': ['Medium (Moderately health-conscious)',
    'Low (Not very concerned)',
    'High (Very health-conscious)'],
    'typical_consumption_situations': ['Active (eg. Sports, gym)',
    'Social (eg. Parties)',
    'Casual (eg. At home)'],
    'price_range': ['100-150', '200-250', '150-200', '50-100']}

len(df[df['zone'] == 'Metor'])

4

df['zone'] = df['zone'].replace({'urbna': 'Urban', 'Metor': 'Metro'})
df['current_brand'] = df['current_brand'].replace({'Establishd':
'Established', 'newcomer': 'Newcomer'})

print(df['zone'].unique() )
print(df['current_brand'].unique())

['Urban' 'Metro' 'Rural' 'Semi-Urban']
['Newcomer' 'Established']

```

Handle missing value

```
df[df.isna().any(axis=1)]
```

	respondent_id	age	gender	zone	occupation	
income_levels \						
4	R00005	23	M	Metro	Student	NaN
5	R00006	22	F	Urban	Student	NaN
11	R00012	23	F	Urban	Student	NaN

14	R00015	22	M	Metro	Student	NaN
20	R00021	23	M	Semi-Urban	Student	NaN
...	...	...	...	...	...	...
29995	R29986	24	F	Metro	Student	NaN
29996	R29987	22	M	Urban	Student	NaN
29999	R29990	23	F	Metro	Student	NaN
30002	R29993	18	M	Metro	Student	NaN
30009	R30000	26	M	Metro	Student	NaN
consume_frequency(weekly)			current_brand			
preferable_consumption_size \						
4	3-4 times		Established		Medium (500	
ml)						
5	5-7 times		Established		Large (1	
L)						
11	0-2 times		Established		Small (250	
ml)						
14	3-4 times		Established		Small (250	
ml)						
20	3-4 times		Newcomer		Large (1	
L)						
...	...		...			
...						
29995	5-7 times		Newcomer		Small (250	
ml)						
29996	0-2 times		Established		Small (250	
ml)						
29999	3-4 times		Established		Small (250	
ml)						
30002	0-2 times		Established		Medium (500	
ml)						
30009	5-7 times		Newcomer		Medium (500	
ml)						
awareness_of_other_brands			reasons_for_choosing_brands			
flavor_preference \						
4	0 to 1		Availability			
Traditional						
5	2 to 4		Price			
Traditional						
11	0 to 1		Price			
Exotic						

14	0 to 1	Price
Traditional		
20	0 to 1	Availability
Traditional		
...	...	...
...		
29995	0 to 1	Price
Traditional		
29996	0 to 1	Price
Exotic		
29999	2 to 4	Price
Exotic		
30002	2 to 4	Availability
Exotic		
30009	above 4	Price
Traditional		

	purchase_channel	packaging_preference \
4	Online	Premium
5	Online	Simple
11	Retail Store	Premium
14	Retail Store	Simple
20	Retail Store	Premium
...	...	...
29995	Online	Premium
29996	Online	Premium
29999	Online	Simple
30002	Online	Premium
30009	Retail Store	Simple

	health_concerns	
typical_consumption_situations \		
4	Medium (Moderately health-conscious)	Active (eg. Sports, gym)
5	Low (Not very concerned)	Active (eg. Sports, gym)
11	Low (Not very concerned)	Social (eg. Parties)
14	Medium (Moderately health-conscious)	Active (eg. Sports, gym)
20	Low (Not very concerned)	Active (eg. Sports, gym)
...	...	...
...		
29995	Low (Not very concerned)	Social (eg. Parties)
29996	Low (Not very concerned)	Social (eg. Parties)
29999	Medium (Moderately health-conscious)	Active (eg. Sports, gym)

gym)		
30002	Low (Not very concerned)	Social (eg.
Parties)		
30009	High (Very health-conscious)	Social (eg.
Parties)		

	price_range
4	50-100
5	100-150
11	50-100
14	100-150
20	50-100
...	...
29995	100-150
29996	50-100
29999	100-150
30002	100-150
30009	150-200

[8070 rows x 17 columns]

df.isna().sum()

respondent_id	0
age	0
gender	0
zone	0
occupation	0
income_levels	8060
consume_frequency(weekly)	8
current_brand	0
preferable_consumption_size	0
awareness_of_other_brands	0
reasons_for_choosing_brands	0
flavor_preference	0
purchase_channel	10
packaging_preference	0
health_concerns	0
typical_consumption_situations	0
price_range	0
dtype: int64	

```
print(df['consume_frequency(weekly)'].mode()[0])
print(df['purchase_channel'].mode()[0])
```

3-4 times
Online

```
df['consume_frequency(weekly)'] =
df['consume_frequency(weekly)'].fillna(df['consume_frequency(weekly)']
.mode()[0])
df['purchase_channel'] =
df['purchase_channel'].fillna(df['purchase_channel'].mode()[0])
df.isna().sum()
```

```
respondent_id      0
age                0
gender             0
zone               0
occupation         0
income_levels      8060
consume_frequency(weekly)  0
current_brand      0
preferable_consumption_size  0
awareness_of_other_brands  0
reasons_for_choosing_brands  0
flavor_preference  0
purchase_channel   0
packaging_preference  0
health_concerns    0
typical_consumption_situations  0
price_range        0
dtype: int64
```

```
missing_df = df[["zone","occupation","income_levels","price_range"]]
missing_df.head(10)
```

	zone	occupation	income_levels	price_range
0	Urban	Working Professional	<10L	100-150
1	Metro	Working Professional	> 35L	200-250
2	Rural	Working Professional	> 35L	200-250
3	Urban	Working Professional	16L - 25L	150-200
4	Metro	Student	NaN	50-100
5	Urban	Student	NaN	100-150
6	Urban	Entrepreneur	10L - 15L	200-250
7	Urban	Entrepreneur	26L - 35L	200-250
8	Semi-Urban	Working Professional	<10L	100-150
9	Semi-Urban	Entrepreneur	10L - 15L	200-250

```
df['income_levels'] = df['income_levels'].fillna('Not Reported')
df.isna().sum()
```

```
respondent_id      0
age                0
gender             0
zone               0
occupation         0
income_levels      0
```

```

consume_frequency(weekly)      0
current_brand                   0
preferable_consumption_size    0
awareness_of_other_brands      0
reasons_for_choosing_brands    0
flavor_preference               0
purchase_channel                0
packaging_preference            0
health_concerns                 0
typical_consumption_situations  0
price_range                     0
dtype: int64

# cleaned_df = df.copy()
# cleaned_df.to_csv('cleaned_survey_results.csv', index=False)

```

Feature Engineering

```

new_df = pd.read_csv('resources/cleaned_survey_results.csv')
new_df.shape

(29991, 17)

new_df.isna().sum()

respondent_id      0
age                0
gender             0
zone               0
occupation          0
income_levels      0
consume_frequency(weekly)  0
current_brand      0
preferable_consumption_size  0
awareness_of_other_brands  0
reasons_for_choosing_brands  0
flavor_preference   0
purchase_channel    0
packaging_preference  0
health_concerns     0
typical_consumption_situations  0
price_range         0
dtype: int64

```

Step 1: Categorize Age into Age Groups

```

def categorize_age(age):
    if 18 <= age <= 25:

```

```

        return '18-25'
    elif 26 <= age <= 35:
        return '26-35'
    elif 36 <= age <= 45:
        return '36-45'
    elif 46 <= age <= 55:
        return '46-55'
    elif 56 <= age <= 70:
        return '56-70'
    else:
        return '70+'

new_df['age'] = new_df['age'].apply(categorize_age)
new_df['age'].value_counts()

age
18-25    10468
26-35     9093
36-45     5972
46-55     2966
56-70     1492
Name: count, dtype: int64

```

Step 2: Create cf_ab_score (Consume Frequency and Awareness Brand Score)

```

new_df['consume_frequency(weekly)'].value_counts()

consume_frequency(weekly)
3-4 times    11794
5-7 times     9774
0-2 times     8423
Name: count, dtype: int64

def frequency_score(freq):
    if freq == '0-2 times':
        return 1
    elif freq == '3-4 times':
        return 2
    else:
        return 3

new_df['frequency_score'] =
new_df['consume_frequency(weekly)'].apply(frequency_score)
new_df[['consume_frequency(weekly)', 'frequency_score']].head(10)

consume_frequency(weekly)  frequency_score
0          3-4 times          2
1          5-7 times          3
2          3-4 times          2
3          5-7 times          3
4          3-4 times          2

```

5	5-7 times	3
6	5-7 times	3
7	5-7 times	3
8	5-7 times	3
9	5-7 times	3

```

new_df['awareness_of_other_brands'].value_counts()

awareness_of_other_brands
0 to 1      12966
2 to 4      11225
above 4      5800
Name: count, dtype: int64

def awareness_score(awareness):
    if awareness == '0 to 1':
        return 1
    elif awareness == '2 to 4':
        return 2
    elif awareness == 'above 4':
        return 3
new_df['awareness_score'] =
new_df['awareness_of_other_brands'].apply(awareness_score)
new_df[['awareness_of_other_brands', 'awareness_score']].head(10)

  awareness_of_other_brands  awareness_score
0              0 to 1                1
1              2 to 4                2
2              2 to 4                2
3              0 to 1                1
4              0 to 1                1
5              2 to 4                2
6              2 to 4                2
7              above 4                3
8              2 to 4                2
9              2 to 4                2

cf_ab_score = new_df['frequency_score'] / (new_df['awareness_score'] +
new_df['frequency_score'])
new_df['cf_ab_score'] = round(cf_ab_score, 2)
new_df['cf_ab_score'].max()

np.float64(0.75)

```

Step 3: Create Zone Affluence Score (ZAS)

```
df['zone'].value_counts()
```

zone	
Metro	11910
Urban	10691

```
Semi-Urban    5274
Rural         2116
Name: count, dtype: int64
```

```
def zone_map(zone):
    if zone == 'Urban':
        return 3
    elif zone == 'Metro':
        return 4
    elif zone == 'Rural':
        return 1
    elif zone == 'Semi-Urban':
        return 2
new_df['zone_score'] = new_df['zone'].apply(zone_map)
new_df[['zone', 'zone_score']].head(10)
```

	zone	zone_score
0	Urban	3
1	Metro	4
2	Rural	1
3	Urban	3
4	Metro	4
5	Urban	3
6	Urban	3
7	Urban	3
8	Semi-Urban	2
9	Semi-Urban	2

```
df['income_levels'].value_counts()
```

```
income_levels
Not Reported    8060
16L - 25L       5897
10L - 15L       5251
<10L            4661
26L - 35L       3872
> 35L           2250
Name: count, dtype: int64
```

```
def categorize_income_levels(income):
    if income == 'Not Reported':
        return 0
    elif income == '16L - 25L':
        return 3
    elif income == '10L - 15L':
        return 2
    elif income == '<10L':
        return 1
    elif income == '26L - 35L':
        return 4
```

```

        elif income == '> 35L':
            return 5
new_df['income_level_score'] =
new_df['income_levels'].apply(categorize_income_levels)
new_df[['income_levels', 'income_level_score']].head(10)

```

	income_levels	income_level_score
0	<10L	1
1	> 35L	5
2	> 35L	5
3	16L - 25L	3
4	Not Reported	0
5	Not Reported	0
6	10L - 15L	2
7	26L - 35L	4
8	<10L	1
9	10L - 15L	2

```

new_df['zas_score'] = new_df['zone_score'] *
new_df['income_level_score']
len(new_df['zas_score'].unique())

```

14

Step 4: Brand Switching Indicator (BSI)

```

new_df['bsi'] = (
    ((new_df['current_brand'] != 'Established') &
     (new_df['reasons_for_choosing_brands'].isin(['Price',
'Quality'])))
    .astype(int)
)
new_df[['bsi', 'reasons_for_choosing_brands', 'current_brand']].head(10)

```

	bsi	reasons_for_choosing_brands	current_brand
0	1	Price	Newcomer
1	0	Quality	Established
2	0	Availability	Newcomer
3	0	Brand Reputation	Newcomer
4	0	Availability	Established
5	0	Price	Established
6	1	Price	Newcomer
7	0	Brand Reputation	Established
8	0	Availability	Newcomer
9	1	Price	Newcomer

```

occupation_age_table = pd.crosstab(new_df['age'],
new_df['occupation'])
print(occupation_age_table)

```

occupation	Entrepreneur	Retired	Student	Working	Professional
age					
18-25	535	0	7328		2605
26-35	1826	0	697		6570
36-45	1619	0	0		4353
46-55	799	0	0		2167
56-70	221	1130	35		106

```

new_df = new_df.drop(
    new_df[(new_df['age'] == '56-70') & (new_df['occupation'] ==
'Student')].index
)

```

```
new_df.shape
```

```
(29956, 24)
```

```
new_df.head()
```

	respondent_id	age	gender	zone	occupation
income_levels \					
0	R00001	26-35	M	Urban	Working Professional
<10L					
1	R00002	46-55	F	Metro	Working Professional
35L					
2	R00003	36-45	F	Rural	Working Professional
35L					
3	R00004	26-35	F	Urban	Working Professional
25L					
4	R00005	18-25	M	Metro	Student
Not Reported					

	consume_frequency(weekly)	current_brand	preferable_consumption_size
\			
0	3-4 times	Newcomer	Medium (500 ml)
1	5-7 times	Established	Medium (500 ml)
2	3-4 times	Newcomer	Medium (500 ml)
3	5-7 times	Newcomer	Medium (500 ml)
4	3-4 times	Established	Medium (500 ml)

awareness_of_other_brands	...	health_concerns
\		

0	0 to 1	...	Medium (Moderately health-conscious)
1	2 to 4	...	Medium (Moderately health-conscious)
2	2 to 4	...	Medium (Moderately health-conscious)
3	0 to 1	...	Low (Not very concerned)
4	0 to 1	...	Medium (Moderately health-conscious)

	typical_consumption_situations	price_range	frequency_score
0	Active (eg. Sports, gym)	100-150	2
1			
1	Social (eg. Parties)	200-250	3
2			
2	Active (eg. Sports, gym)	200-250	2
2			
3	Active (eg. Sports, gym)	150-200	3
1			
4	Active (eg. Sports, gym)	50-100	2
1			

	cf_ab_score	zone_score	income_level_score	zas_score	bsi
0	0.67	3	1	3	1
1	0.60	4	5	20	0
2	0.50	1	5	5	0
3	0.75	3	3	9	0
4	0.67	4	0	0	0

[5 rows x 24 columns]

```
len(new_df[new_df['bsi'] == 0])
```

20796

Modeling

```
from sklearn.model_selection import train_test_split
```

Prepare Feature and Target Variables

```
new_df.shape
```

```
(29956, 24)
```

```
X = new_df.drop(columns=['respondent_id', 'price_range'])
y = new_df['price_range']
```

Data Splitting

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.25, random_state=42)
print("X_train shape",X_train.shape)
print("X_test shape",X_test.shape)
print("y_train shape",y_train.shape)
print("y_test shape",y_test.shape)
```

```
X_train shape (22467, 22)
```

```
X_test shape (7489, 22)
```

```
y_train shape (22467,)
```

```
y_test shape (7489,)
```

```
train_df = pd.concat([X_train, y_train], axis=1)
```

```
test_df = pd.concat([X_test, y_test], axis=1)
```

```
print("Train DataFrame shape:", train_df.shape)
```

```
print("Test DataFrame shape:", test_df.shape)
```

```
Train DataFrame shape: (22467, 23)
```

```
Test DataFrame shape: (7489, 23)
```

Feature Encoding

```
categorical_cols =
X.select_dtypes(include=['object','category']).columns
numerical_cols =
X.select_dtypes(exclude=['object','category']).columns
print("Categorical columns:", categorical_cols.tolist())
print("Number of Categorical columns:",
len(categorical_cols.tolist()))
print("Numerical columns:", numerical_cols.tolist())
print("Number of Numerical columns:", len(numerical_cols.tolist()))
```

```
Categorical columns: ['age', 'gender', 'zone', 'occupation',
'income_levels', 'consume_frequency(weekly)', 'current_brand',
'preferable_consumption_size', 'awareness_of_other_brands',
'reasons_for_choosing_brands', 'flavor_preference',
'purchase_channel', 'packaging_preference', 'health_concerns',
'typical_consumption_situations']
```

```
Number of Categorical columns: 15
```

```
Numerical columns: ['frequency_score', 'awareness_score',
'cf_ab_score', 'zone_score', 'income_level_score', 'zas_score', 'bsi']
```

```
Number of Numerical columns: 7
```

```
from sklearn.preprocessing import LabelEncoder
```

Level Encoding

```
level_encoding_cols = ['age', 'income_levels', 'health_concerns',
                        'consume_frequency(weekly)', 'preferable_consumption_size',
                        'price_range']
```

```
le = LabelEncoder()
```

```
for col in level_encoding_cols:
    train_df[col] = le.fit_transform(train_df[col])
    test_df[col] = le.transform(test_df[col])
```

```
train_df.head()
```

	age	gender	zone	occupation	income_levels	\
8385	0	F	Semi-Urban	Working Professional	0	
26377	0	F	Urban	Working Professional	1	
11269	0	F	Urban	Student	5	
9103	1	M	Metro	Working Professional	1	
3370	1	M	Metro	Entrepreneur	1	

	consume_frequency(weekly)	current_brand
8385	0	Newcomer
2		
26377	2	Established
1		
11269	1	Newcomer
2		
9103	1	Established
0		
3370	0	Established
0		

	awareness_of_other_brands	reasons_for_choosing_brands	...	\
8385	above 4	Availability	...	
26377	2 to 4	Quality	...	
11269	2 to 4	Price	...	
9103	2 to 4	Quality	...	
3370	above 4	Brand Reputation	...	

	health_concerns	typical_consumption_situations	
8385	1	Social (eg. Parties)	1
26377	2	Social (eg. Parties)	3
11269	2	Active (eg. Sports, gym)	2
9103	1	Active (eg. Sports, gym)	2
3370	2	Casual (eg. At home)	1

awareness_score	cf_ab_score	zone_score	income_level_score
-----------------	-------------	------------	--------------------

zas_score \				
8385	3	0.25	2	2
4				
26377	2	0.60	3	3
9				
11269	2	0.50	3	0
0				
9103	2	0.50	4	3
12				
3370	3	0.25	4	3
12				

	bsi	price_range
8385	0	3
26377	0	2
11269	1	0
9103	0	1
3370	0	1

[5 rows x 23 columns]

One-Hot Encoding

```
# one_hot_encoded.shape
one_hot_encoded_cols = [col for col in categorical_cols.tolist() if
col not in level_encoding_cols]
```

```
train_df = pd.get_dummies(train_df, columns=one_hot_encoded_cols,
drop_first=True)
test_df = pd.get_dummies(test_df, columns=one_hot_encoded_cols,
drop_first=True)
```

```
train_df.head()
```

	age	income_levels	consume_frequency(weekly) \	
8385	0	0	0	
26377	0	1	2	
11269	0	5	1	
9103	1	1	1	
3370	1	1	0	

	preferable_consumption_size	health_concerns	
frequency_score \			
8385	2	1	1
26377	1	2	3
11269	2	2	2

9103	0	1	2
3370	0	2	1

	awareness_score	cf_ab_score	zone_score
income_level_score ... \			
8385	3	0.25	2
2 ...			
26377	2	0.60	3
3 ...			
11269	2	0.50	3
0 ...			
9103	2	0.50	4
3 ...			
3370	3	0.25	4
3 ...			

	awareness_of_other_brands_above 4 \
8385	True
26377	False
11269	False
9103	False
3370	True

	reasons_for_choosing_brands_Brand Reputation \
8385	False
26377	False
11269	False
9103	False
3370	True

	reasons_for_choosing_brands_Price
reasons_for_choosing_brands_Quality \	
8385	False
False	
26377	False
True	
11269	True
False	
9103	False
True	
3370	False
False	

	flavor_preference_Traditional	purchase_channel_Retail	Store \
8385	True		False
26377	True		False
11269	True		False
9103	True		False

3370	True	False
	packaging_preference_Premium	packaging_preference_Simple \
8385	False	True
26377	True	False
11269	False	True
9103	True	False
3370	False	False

	typical_consumption_situations_Casual (eg. At home) \
8385	False
26377	False
11269	False
9103	False
3370	True

	typical_consumption_situations_Social (eg. Parties)
8385	True
26377	True
11269	False
9103	False
3370	False

[5 rows x 32 columns]

test_df.head()

	age	income_levels	consume_frequency(weekly)	\
11973	0	5	2	
1805	0	0	0	
517	1	1	0	
3817	3	1	2	
28607	2	2	2	

	preferable_consumption_size	health_concerns
frequency_score \		
11973	0	2 3
1805	2	0 1
517	1	2 1
3817	1	2 3
28607	1	0 3

	awareness_score	cf_ab_score	zone_score
income_level_score ... \			
11973	1	0.75	3
0 ...			

1805	3	0.25	4
2 ...			
517	3	0.25	4
3 ...			
3817	2	0.60	4
3 ...			
28607	1	0.75	4
4 ...			

	awareness_of_other_brands_above 4 \
11973	False
1805	True
517	True
3817	False
28607	False

	reasons_for_choosing_brands_Brand Reputation \
11973	False
1805	False
517	True
3817	False
28607	False

	reasons_for_choosing_brands_Price
reasons_for_choosing_brands_Quality \	
11973	True
False	
1805	True
False	
517	False
False	
3817	False
True	
28607	False
True	

	flavor_preference_Traditional	purchase_channel_Retail	Store \
11973	False		False
1805	False		False
517	False		True
3817	True		True
28607	False		False

	packaging_preference_Premium	packaging_preference_Simple \
11973	False	True
1805	False	True
517	True	False
3817	True	False
28607	True	False

	typical_consumption_situations_Casual (eg. At home) \
11973	False
1805	False
517	False
3817	False
28607	True

	typical_consumption_situations_Social (eg. Parties)
11973	True
1805	False
517	False
3817	False
28607	False

[5 rows x 32 columns]

Model Selection:

```
# ML imports
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

import xgboost as xgb
import lightgbm as lgb

# Define features and target
X_train = train_df.drop(columns=["price_range"]) # replace with your
actual target
y_train = train_df["price_range"]

X_test = test_df.drop(columns=["price_range"])
y_test = test_df["price_range"]

print("Train:", y_train.value_counts())
print("Test:", y_test.value_counts())
```

```
Train: price_range
2    7281
1    6574
0    5863
3    2749
Name: count, dtype: int64
Test: price_range
2    2430
1    2223
```

```
0    1930
3     906
Name: count, dtype: int64
```

```
# Dictionary of models
```

```
models = {
    "GaussianNB": GaussianNB(),
    "LogisticRegression": LogisticRegression(),
    "SVM": SVC(kernel="linear", probability=True),
    "RandomForest": RandomForestClassifier(),
    "XGBoost": xgb.XGBClassifier(eval_metric="logloss"),
    "LightGBM": lgb.LGBMClassifier()
}
```

```
# Train and evaluate each model
```

```
for name, model in models.items():
    print(f"\n Training {name}...")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    print(f" Accuracy of {name}: {acc:.4f}")
    print(f" Classification Report for {name}:\n")
    print(classification_report(y_test, y_pred))
    print("-" * 60)
```

```
 Training GaussianNB...
```

```
 Accuracy of GaussianNB: 0.5825
```

```
 Classification Report for GaussianNB:
```

	precision	recall	f1-score	support
0	0.47	0.30	0.36	1930
1	0.57	0.39	0.46	2223
2	0.72	0.87	0.79	2430
3	0.45	0.90	0.60	906
accuracy			0.58	7489
macro avg	0.55	0.61	0.55	7489
weighted avg	0.58	0.58	0.56	7489

```
-----
 Training LogisticRegression...
```

```
/opt/anaconda3/envs/exp_venv/lib/python3.12/site-packages/sklearn/
linear_model/_logistic.py:473: ConvergenceWarning: lbfgs failed to
converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Increase the number of iterations to improve the convergence (max_iter=100).

You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

□ Accuracy of LogisticRegression: 0.8044

□ Classification Report for LogisticRegression:

	precision	recall	f1-score	support
0	0.74	0.76	0.75	1930
1	0.76	0.78	0.77	2223
2	0.91	0.89	0.90	2430
3	0.79	0.72	0.75	906
accuracy			0.80	7489
macro avg	0.80	0.79	0.79	7489
weighted avg	0.81	0.80	0.80	7489

□ Training SVM...

□ Accuracy of SVM: 0.8117

□ Classification Report for SVM:

	precision	recall	f1-score	support
0	0.75	0.76	0.76	1930
1	0.77	0.78	0.78	2223
2	0.91	0.90	0.90	2430
3	0.79	0.75	0.77	906
accuracy			0.81	7489
macro avg	0.81	0.80	0.80	7489
weighted avg	0.81	0.81	0.81	7489

□ Training RandomForest...

□ Accuracy of RandomForest: 0.8987

□ Classification Report for RandomForest:

	precision	recall	f1-score	support
0	0.89	0.87	0.88	1930
1	0.85	0.89	0.87	2223

2	0.94	0.93	0.94	2430
3	0.92	0.89	0.90	906
accuracy			0.90	7489
macro avg	0.90	0.90	0.90	7489
weighted avg	0.90	0.90	0.90	7489

□ Training XGBoost...
 □ Accuracy of XGBoost: 0.9263
 □ Classification Report for XGBoost:

	precision	recall	f1-score	support
0	0.92	0.91	0.91	1930
1	0.90	0.92	0.91	2223
2	0.96	0.95	0.96	2430
3	0.92	0.92	0.92	906
accuracy			0.93	7489
macro avg	0.92	0.92	0.92	7489
weighted avg	0.93	0.93	0.93	7489

□ Training LightGBM...
 [LightGBM] [Warning] Found whitespace in feature_names, replace with underlines
 [LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000618 seconds.
 You can set `force\_row\_wise=true` to remove the overhead.
 And if memory is not enough, you can set `force\_col\_wise=true`.
 [LightGBM] [Info] Total Bins 101
 [LightGBM] [Info] Number of data points in the train set: 22467, number of used features: 31
 [LightGBM] [Info] Start training from score -1.343386
 [LightGBM] [Info] Start training from score -1.228925
 [LightGBM] [Info] Start training from score -1.126779
 [LightGBM] [Info] Start training from score -2.100810
 □ Accuracy of LightGBM: 0.9248
 □ Classification Report for LightGBM:

	precision	recall	f1-score	support
0	0.91	0.90	0.91	1930
1	0.90	0.91	0.91	2223
2	0.96	0.96	0.96	2430
3	0.92	0.92	0.92	906

accuracy			0.92	7489
macro avg	0.92	0.92	0.92	7489
weighted avg	0.92	0.92	0.92	7489

MLFlow Deployment and Dagshub Publishing

```
import mlflow
from sklearn.metrics import accuracy_score, classification_report
import dagshub

dagshub.init(repo_owner='Aadarsh4u-code', repo_name='Survey-price-range-prediction', mlflow=True)
mlflow.set_tracking_uri("https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow")
mlflow.set_experiment("survey_price_range_classification")

models = {
    "GaussianNB": GaussianNB(),
    "LogisticRegression": LogisticRegression(max_iter=1000),
    "SVM": SVC(kernel="linear", probability=True),
    "RandomForest": RandomForestClassifier(n_estimators=200,
    random_state=42),
    "XGBoost": xgb.XGBClassifier(use_label_encoder=False,
eval_metric="logloss"),
    "LightGBM": lgb.LGBMClassifier()
}
```

⚠ AUTHORIZATION REQUIRED ⚠

```
/opt/anaconda3/envs/exp_venv/lib/python3.12/site-packages/rich/
live.py:256: UserWarning: install "ipywidgets" for
Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

Open the following link in your browser to authorize the client:
https://dagshub.com/login/oauth/authorize?state=382a5d1a-f938-4992-9864-0d98a788738f&client_id=32b60ba385aa7cecf24046d8195a71c07dd345d9657977863b52e7748e0f0f28&middleman_request_id=ec3d1071ebb3d05abb1e44a1b6a062b93f2909589ecee956d67b3b2e9d84655d

Accessing as Aadarsh4u-code

Initialized MLflow to track repo "Aadarsh4u-code/Survey-price-range-prediction"

Repository Aadarsh4u-code/Survey-price-range-prediction initialized!

2025/09/27 21:31:57 INFO mlflow.tracking.fluent: Experiment with name 'survey_price_range_classification' does not exist. Creating a new experiment.

```
for name, model in models.items():
    with mlflow.start_run(run_name=name):
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        # Log metrics
        acc = accuracy_score(y_test, y_pred)
        mlflow.log_metric("accuracy", acc)

        # Optionally log more metrics like F1, precision, recall
        report = classification_report(y_test, y_pred,
output_dict=True)
        mlflow.log_metric("precision_macro", report["macro avg"]
["precision"])
        mlflow.log_metric("recall_macro", report["macro avg"]
["recall"])
        mlflow.log_metric("f1_macro", report["macro avg"]["f1-score"])

        # Log model
        mlflow.sklearn.log_model(model, "model")

        print(f"Logged {name} with accuracy: {acc:.4f}")
```

2025/09/27 21:32:16 WARNING mlflow.models.model: Model logged without a signature and input example. Please set `input\_example` parameter when logging the model to auto infer the model signature.

Logged GaussianNB with accuracy: 0.5825

□ View run GaussianNB at: <https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0/runs/a7efcba9f2c449659f89c1fc0c103632>

□ View experiment at: <https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0>

2025/09/27 21:32:30 WARNING mlflow.models.model: Model logged without a signature and input example. Please set `input\_example` parameter when logging the model to auto infer the model signature.

Logged LogisticRegression with accuracy: 0.8100

□ View run LogisticRegression at:

<https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0/runs/22d4118fdd1244048133cd3b1ad78205>

□ View experiment at: <https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0>

2025/09/27 21:33:34 WARNING mlflow.models.model: Model logged without a signature and input example. Please set `input\_example` parameter when logging the model to auto infer the model signature.

Logged SVM with accuracy: 0.8117

□ View run SVM at: <https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0/runs/a95fb2e91c2148f783acbda90d9b3828>

□ View experiment at: <https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0>

2025/09/27 21:33:49 WARNING mlflow.models.model: Model logged without a signature and input example. Please set `input\_example` parameter when logging the model to auto infer the model signature.

Logged RandomForest with accuracy: 0.8996

□ View run RandomForest at: <https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0/runs/96677ce0b31f49dab97a8c208b1b478b>

□ View experiment at: <https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0>

/opt/anaconda3/envs/exp_venv/lib/python3.12/site-packages/xgboost/training.py:183: UserWarning: [21:34:22] WARNING:
/Users/runner/work/xgboost/xgboost/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

bst.update(dtrain, iteration=i, fobj=obj)

2025/09/27 21:34:27 WARNING mlflow.models.model: Model logged without a signature and input example. Please set `input\_example` parameter when logging the model to auto infer the model signature.

Logged XGBoost with accuracy: 0.9263

□ View run XGBoost at: <https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0/runs/e071a469a5794fce82032c5cef9ccf36>

□ View experiment at: <https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0>

[LightGBM] [Warning] Found whitespace in feature_names, replace with underlines

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000244 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 101

```
[LightGBM] [Info] Number of data points in the train set: 22467,  
number of used features: 31  
[LightGBM] [Info] Start training from score -1.343386  
[LightGBM] [Info] Start training from score -1.228925  
[LightGBM] [Info] Start training from score -1.126779  
[LightGBM] [Info] Start training from score -2.100810
```

```
2025/09/27 21:34:37 WARNING mlflow.models.model: Model logged without  
a signature and input example. Please set `input_example` parameter  
when logging the model to auto infer the model signature.
```

```
Logged LightGBM with accuracy: 0.9248
```

```
□ View run LightGBM at: https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0/runs/799afa3782ec4b3984c3f15f7907acb3
```

```
□ View experiment at: https://dagshub.com/Aadarsh4u-code/Survey-price-range-prediction.mlflow/#/experiments/0
```